Roadmap técnico de desarrollo Panel Gradu

Especificación funcional, técnica y de despliegue
para construcción modular con Antigravity

Versión borrador - Abril 2026

Repositorio obligatorio	GitHub: panel-gradu
Stack base	React + Node.js/Express + MariaDB
Infraestructura objetivo	Docker local y producción detrás de Traefik
Dominio previsto	gradu.hxl.red o subdominio equivalente de HxL
Objetivo principal	Gestión integral de graduación, cuotas, rifas, finanzas, cantina y control operativo

Propósito del documento

Este documento debe servir como base de ejecución para Antigravity. No es una presentación comercial ni una explicación para usuarios finales: define alcance, módulos, reglas de negocio, entregables y criterios de aceptación con el nivel de detalle suficiente para evitar ambigüedades durante la construcción.

1. Objetivo del sistema y contexto de negocio

Panel Gradu es una plataforma web interna para la organización de la graduación. Su función es centralizar en un único sistema la información operativa que hoy se gestiona de forma distribuida en archivos Excel y comunicación manual entre organizadores. El sistema debe resolver problemas de trazabilidad, consistencia de datos, visibilidad del estado financiero, seguimiento de cuotas, control de rifas, planificación de cantina y coordinación por clase.

El uso inicial está restringido a organizadores. No obstante, la arquitectura debe dejar preparado un esquema de expansión futura para habilitar acceso de estudiantes en modo consulta parcial, sin rediseñar la base del producto. La primera versión no debe implementar venta de entradas, dado que ese proceso depende de un sistema externo provisto por la empresa organizadora del evento.

1.1 Restricciones de base

- Base de datos obligatoria: MySQL o MariaDB; para este proyecto se define MariaDB.
- Desarrollo y prueba local obligatorios mediante Docker Compose, sin depender de infraestructura externa.
- Repositorio Git obligatorio con nombre panel-gradu.
- Despliegue manual en servidor HxL; no se requiere CI/CD automatizado en la primera etapa.
- Exposición web detrás de Traefik, con HTTPS y routing por subdominio.
- Moneda única del sistema: pesos uruguayos, almacenados sin decimales.
- Los identificadores internos son exclusivamente técnicos; en frontend deben prevalecer nombre, clase, turno y métricas operativas comprensibles.

1.2 Alcance funcional incluido

- Gestión de usuarios internos con autenticación por usuario y contraseña.
- Gestión de roles base: admin, organizer y student.
- Relación organizador-clase para determinar responsabilidad operativa sobre una o más clases.
- Alta, edición, baja lógica y consulta de estudiantes.
- Configuración mensual de cuotas y registro manual de pagos.
- Registro de cobertura parcial o total mediante rifas, con separación entre monto aplicado a cuota y excedente hacia fondo común.
- Registro de entrega y devolución de rifas por estudiante, incluyendo fechas y montos.
- Libro contable básico de ingresos y egresos por categoría.
- Dashboard con métricas, desgloses y proyección futura.
- Calendario operativo para cantina, hitos de pago y eventos internos.
- Auditoría completa de acciones realizadas en el sistema.

1.3 Fuera de alcance en MVP

- Venta de entradas y emisión de tickets.
- Pasarelas de pago o cobros online automáticos.
- Control granular de permisos por acción o por vista.

- Integración con WhatsApp, correo, Discord u otros canales.
- Módulo de inventario detallado para cantina o stock avanzado.
- Aplicación móvil nativa.

2. Arquitectura objetivo

La arquitectura propuesta es una aplicación web desacoplada en frontend y backend, con base de datos relacional y despliegue containerizado. La elección prioriza portabilidad, mantenibilidad, facilidad de prueba local y compatibilidad con la infraestructura actual de HxL.

2.1 Componentes

- Frontend SPA en React, orientado a consumo de API REST y a operación intensiva de panel administrativo.
- Backend Node.js con Express, responsable de autenticación, lógica de negocio, validación, auditoría y acceso a datos.
- Base MariaDB en contenedor dedicado del proyecto.
- Nginx opcional dentro del contenedor frontend para servir build estático; alternativamente, build servida por el propio contenedor frontend con configuración simple.
- Traefik externo como reverse proxy de producción.

2.2 Entornos

Entorno	Objetivo	Requisitos mínimos
Local	Desarrollo y test funcional	Docker Compose, variables .env, puertos locales, seed opcional
Staging opcional	Validación previa a producción	No obligatorio en primera fase
Producción	Uso operativo por organizadores	Traefik, subdominio, persistencia DB, backups, HTTPS

2.3 Estructura de repositorio sugerida

```
panel-gradu/
  backend/
    src/
      modules/
      middleware/
      lib/
      db/
      routes/
      jobs/
    Dockerfile
  frontend/
    src/
      pages/
      components/
```

```
      hooks/
      services/
      layouts/
    Dockerfile
  infra/
    docker-compose.yml
    .env.example
    traefik-example.yml
  docs/
    roadmap/
    api/
    data-model/
```

3. Modelo operativo y entidades principales

El modelo de datos debe construirse de forma explícita, relacional y auditable. Las entidades principales no deben acoplarse al frontend. Cada módulo debe operar sobre reglas de negocio centradas en estudiantes, clases, períodos mensuales y movimientos de dinero.

3.1 Clases y turnos

Las clases conocidas al inicio del proyecto son las siguientes:

Turno mañana	Turno tarde
Artístico 1	Artístico 2
Derecho 1	Derecho 3
Derecho 2	Economía
Derecho 3	Medicina 3
Medicina 1	Medicina 4
Medicina 2	Ingeniería 2
Ingeniería 1	

La existencia de 'Derecho 3' en ambos turnos obliga a separar clase y turno como datos distintos, o a modelar una clase académica con identificador propio por combinación nombre-turno. La recomendación es persistir clases como entidades únicas con un campo shift obligatorio.

3.2 Usuarios, estudiantes y bajas lógicas

- users: credenciales internas y rol de acceso.
- classes: catálogo de clases y turno asociado.
- organizer_classes: tabla de unión para múltiples asignaciones por organizador.
- students: nombre, clase, turno, bandera wants_hoodie, estado operativo y deleted_at para soft-delete.
- monthly_fee_config: importe definido para cada mes/año.
- payments: registro de pago por estudiante y período.
- raffle_logs: movimientos de rifas entregadas, devueltas y dinero asociado.
- transactions: libro contable básico de ingresos y egresos.

- events_calendar: agenda operativa.
- audit_logs: registro inmutable de acciones relevantes.

3.3 Convenciones de persistencia

- Todos los importes monetarios se almacenan como INT en pesos uruguayos, sin decimales.
- Las bajas deben ser lógicas mediante deleted_at o campo equivalente; no se permite hard delete en registros operativos.
- Todos los registros críticos deben incluir created_at y updated_at.
- Las tablas sensibles a trazabilidad deben incluir created_by y/o updated_by cuando resulte útil.
- Las tablas de movimientos deben conservar el historial; no se deben sobrescribir hechos pasados.

4. Reglas de negocio no negociables

- La cuota mensual debe poder configurarse por mes y año; no queda hardcodeada en la aplicación.
- El registro de que un estudiante pagó una cuota siempre es manual.
- El método de pago reconocido para la cuota es cash o transfer.
- Las rifas no reemplazan la existencia de la cuota mensual: el sistema debe registrar cuándo un estudiante cubrió el mes con rifas y cuánto excedente fue a fondo común.
- Si un estudiante vende rifas por encima del mínimo, el excedente no se imputa automáticamente a meses futuros; debe computarse como fondo común del mes en curso.
- Si un estudiante se baja de la graduación, lo ya aportado no se devuelve; el sistema debe conservar todo el historial y marcar al estudiante como dropped o inactive según nomenclatura final.
- No existe transporte oficial para invitados y el sistema no debe modelarlo como flujo operativo.
- Los pagos importantes del evento deben poder registrarse en calendario y finanzas: garantía, hitos de julio/octubre y saldo final previo al evento.
- La información visible en paneles y listados debe priorizar nombre del estudiante, clase, turno, estado de cuota y métricas de rifas.

5. Roadmap de desarrollo por módulos

La construcción debe dividirse en módulos cerrados, con entregables verificables. Antigravity no debe intentar resolver toda la plataforma de una vez: cada etapa debe ser funcional, testeable y desplegable por separado. Los módulos se detallan a continuación en orden recomendado.

Módulo 0 - Fundación técnica del proyecto

Crear la base técnica del repositorio, los contenedores, la aplicación mínima y los cimientos de desarrollo local/producción.

Alcance del módulo

- Inicialización del repositorio panel-gradu con backend, frontend e infra separados.
- Dockerfiles para frontend y backend.
- docker-compose para ejecución local con servicios app y db.

- Configuración de variables de entorno y .env.example.
- Conexión del backend a MariaDB con capa de acceso a datos y migraciones.
- Estructura base del frontend con layout administrativo, routing, manejo de sesión y cliente HTTP.
- Healthcheck simple de backend y prueba de conectividad con base.

Vistas o superficies de interfaz

- Pantalla de placeholder segura para login.
- Layout base con sidebar superior o lateral, contenedor de contenido, manejo de loading/error y tema visual coherente.

Criterios de aceptación

- Proyecto levanta en local con un único comando de Docker Compose.
- Backend expone endpoint /health y responde OK.
- Frontend consume API y muestra estado de conectividad.
- Se documenta proceso de setup local en README.

Módulo 1 - Autenticación, usuarios internos y clases

Habilitar acceso autenticado al sistema y construir las entidades maestras mínimas para operar.

Alcance del módulo

- Login por username y password con hash bcrypt.
- JWT o sesión equivalente para autenticación stateless.
- Modelo users con roles admin, organizer y student.
- Modelo classes y seed inicial con todas las clases y turnos definidos.
- Relación organizer_classes para vincular organizadores con una o varias clases.
- Vista administrativa para alta/edición de usuarios internos.
- Soft-delete o desactivación controlada de usuarios cuando corresponda.

Vistas o superficies de interfaz

- Pantalla de login.
- Pantalla de usuarios internos con filtros por rol y estado.
- Pantalla de clases con vista de responsables asignados.

Criterios de aceptación

- Solo usuarios autenticados acceden al panel.
- Es posible crear, editar y desactivar usuarios.
- Es posible asignar clases a organizadores y consultar ese vínculo.
- Las sesiones inválidas redirigen correctamente al login.

Módulo 2 - Gestión de estudiantes

Centralizar el padrón operativo de graduados y permitir filtrado real por clase, turno y estado.

Alcance del módulo

- Entidad students con nombre, class_id, shift, wants_hoodie, estado y soft-delete.
- Alta manual de estudiantes.
- Edición de datos básicos.
- Cambio de estado a dropped sin perder historial.
- Búsqueda por nombre, clase, turno y estado.
- Filtros rápidos para responsables de clase.
- Preparación del modelo para acceso futuro de usuarios student sin activarlo todavía.

Vistas o superficies de interfaz

- Listado principal de estudiantes con filtros y buscador.
- Formulario de alta/edición.
- Detalle resumido del estudiante con resumen operativo.

Criterios de aceptación

- Se puede consultar cuántos estudiantes hay por clase y turno.
- El soft-delete no elimina historial asociado.
- Los campos mínimos solicitados por negocio quedan cubiertos sin datos redundantes.

Módulo 3 - Configuración de cuotas y registro de pagos

Resolver la administración mensual de cuotas y su estado real por estudiante.

Alcance del módulo

- Tabla monthly_fee_config por mes y año.
- Pantalla de configuración mensual del valor de la cuota.
- Generación lógica del período de trabajo abril-diciembre, manteniendo compatibilidad futura con otros años.
- Registro manual de pago por estudiante y período.
- Estado de cuota: pending, paid, covered_by_raffles o nomenclatura equivalente consistente.
- Registro de payment_method: cash o transfer.
- Registro opcional de fecha de depósito, observaciones y monto efectivamente abonado.
- Validaciones para evitar doble cierre accidental del mismo período.

Vistas o superficies de interfaz

- Vista mensual tipo matriz clase/estudiante/estado.
- Formulario rápido para marcar pago.
- Detalle histórico de cuotas por estudiante.

Criterios de aceptación

- Cada mes puede tener un importe distinto configurable.
- Cada estudiante muestra claramente qué meses pagó, cuáles tiene pendientes y cuáles cubrió con rifas.

- No se requieren IDs visibles para operar desde frontend.

Módulo 4 - Gestión integral de rifas

Registrar el circuito real de entrega, devolución y aplicación económica de rifas sin identificador unitario por ticket.

Alcance del módulo

- Registro de entrega de rifas por estudiante, con cantidad y fecha.
- Posibilidad de múltiples entregas parciales dentro de un mismo mes.
- Registro de devolución de rifas vendidas con cantidad, fecha y dinero recolectado.
- Campos para consignar si el efectivo quedó pendiente de depósito o si se cerró en el momento.
- Cálculo del monto aplicado a cuota mensual.
- Cálculo del excedente enviado a fondo común.
- Integración con payments para reflejar cobertura mediante rifas.
- Historial legible para auditar quién recibió, cuánto devolvió y cómo se computó.

Vistas o superficies de interfaz

- Listado de movimientos de rifas por estudiante.
- Vista resumen del mes con rifas entregadas, rifas devueltas y estado de cierre.
- Atajo desde la ficha del estudiante para registrar nueva entrega o devolución.

Criterios de aceptación

- El sistema soporta más de una entrega de rifas al mismo estudiante.
- El excedente nunca se aplica automáticamente a meses siguientes.
- El historial de rifas y el historial de pagos permanecen reconciliables.

Módulo 5 - Finanzas y libro contable básico

Crear una visión financiera consistente de ingresos y egresos sin construir un ERP completo.

Alcance del módulo

- Tabla transactions con tipo income/expense, categoría, monto y descripción.
- Categorías iniciales recomendadas: fees, raffles, common_fund, canteen, events, deposits, guarantees, transport, other.
- Registro manual de movimientos extraordinarios, por ejemplo cantina, torneos, fiestas, rifas particulares o pagos a proveedores.
- Relación lógica, no necesariamente física, entre movimientos contables y módulos operativos.
- Vista de saldo actual, acumulado mensual y desgloses por categoría.
- Posibilidad de registrar pagos importantes del evento como egresos identificables.

Vistas o superficies de interfaz

- Libro diario o listado de movimientos con filtros por fecha, categoría y tipo.
- Resumen superior de saldo total y composición.
- Formulario de alta de movimiento manual.

Criterios de aceptación

- Es posible reconstruir el saldo contable del sistema desde la interfaz.
- Los desgloses separan claramente fondos comunes, cuotas y recaudaciones varias.
- No se pierde la capacidad de conciliar manualmente con la tarjeta/cuenta real.

Módulo 6 - Dashboard, métricas y proyecciones

Transformar datos operativos en visibilidad ejecutiva para decisión rápida.

Alcance del módulo

- Resumen de total recaudado.
- Resumen de fondo común acumulado.
- Resumen de ingresos por cuotas, rifas, cantina, eventos y otras categorías.
- Cantidad de cuotas pendientes globales y por clase.
- Indicadores de estudiantes con atraso.
- Ranking o resumen de rifas por clase o estudiante.
- Proyección futura automática basada en cuotas pendientes configuradas, tendencia histórica de pagos y ritmo de recaudación por categorías.

Vistas o superficies de interfaz

- Dashboard principal con tarjetas KPI.
- Gráficos por mes y por categoría.
- Sección de alertas: meses con baja recaudación, clases con más pendientes, estudiantes sin movimientos recientes.

Criterios de aceptación

- El dashboard no depende de cálculos manuales externos.
- Las métricas coinciden con los datos base del sistema.
- La proyección debe estar claramente etiquetada como estimación y explicar supuestos mínimos.

Módulo 7 - Calendario operativo y cantina

Coordinar tareas manuales recurrentes sin depender de cadenas de mensajes o múltiples planillas paralelas.

Alcance del módulo

- Calendario interno con alta manual de eventos.
- Tipos de evento iniciales: canteen, payment_deadline, internal_event, milestone.
- Registro de qué estudiante, clase u organizador se encarga de un día o turno de cantina.
- Carga de hitos fijos: pagos importantes del evento, fecha límite de ciertos procesos, cierres mensuales de rifas.
- Vista mensual y semanal.

Vistas o superficies de interfaz

- Calendario gráfico navegable.

- Listado alternativo para móviles o revisión rápida.
- Formulario de evento con asignaciones y notas.

Criterios de aceptación

- Se pueden ver rápidamente responsabilidades de cantina y fechas críticas.
- Los eventos se filtran por tipo y por clase.
- El módulo es útil operativamente sin convertirse en un calendario generalista complejo.

Módulo 8 - Auditoría, trazabilidad y herramientas administrativas

Garantizar capacidad de revisión posterior sobre cambios relevantes y brindar estabilidad operativa.

Alcance del módulo

- Tabla audit_logs con actor, acción, entidad, entidad_id, metadata y timestamp.
- Registro de login exitoso, alta/edición/baja de estudiantes, cambios de cuotas, pagos, movimientos de rifas, transacciones financieras y eventos.
- Visor de logs con filtros por usuario, entidad y rango de fechas.
- Herramientas administrativas mínimas: seed inicial, reindexado sencillo si aplica, backup lógico documentado.

Vistas o superficies de interfaz

- Listado de logs con metadata resumida y detalle expandible.
- Indicadores para identificar cambios críticos recientes.

Criterios de aceptación

- Toda operación sensible deja huella auditable.
- Los logs son consultables sin entrar a base de datos manualmente.
- El costo de almacenamiento sigue siendo razonable para el volumen esperado.

Módulo 9 - Apertura futura a estudiantes (fase posterior, no MVP)

Definir la expansión controlada del sistema para consulta parcial de estudiantes sin reestructurar la plataforma.

Alcance del módulo

- Modelo de acceso student ya contemplado desde el inicio.
- Portal o vista acotada para que cada estudiante consulte cuotas pagadas, cuotas pendientes y resumen propio.
- Restricción estricta a datos propios; no se exponen datos globales ni operaciones administrativas.
- Preparación para recuperación de contraseña o mecanismo simple de onboarding futuro.

Vistas o superficies de interfaz

- Home minimalista para estudiante.
- Historial personal de cuotas y estado.

Criterios de aceptación

- Esta fase se planifica pero no bloquea el MVP.
- Nada del diseño actual impide incorporarla después.

6. Requisitos no funcionales

6.1 Usabilidad

- La interfaz debe ser intuitiva y visualmente cuidada; no se busca un panel genérico improvisado.
- La navegación principal debe requerir pocos clics para registrar pagos, rifas o movimientos.
- Los formularios frecuentes deben optimizar velocidad operativa antes que ornamentalidad.
- El sistema debe ser completamente responsive, dado que se usará casi siempre desde teléfonos móviles.

6.2 Calidad técnica

- Código modular y legible, con separación clara entre dominio, acceso a datos y presentación.
- Validaciones consistentes en backend; el frontend no debe ser la única barrera.
- Manejo explícito de errores y mensajes comprensibles para usuario interno.
- Tests mínimos recomendados para utilidades críticas, cálculos de rifas y proyecciones.
- Semillas de datos iniciales documentadas para facilitar arranque de desarrollo.

6.3 Seguridad y operación

- Passwords siempre hasheadas con bcrypt.
- Secrets fuera del repositorio y definidos por variables de entorno.
- Protección básica contra sesiones inválidas, acceso sin token y rutas privadas.
- Backups de base de datos documentados como procedimiento operativo, aunque la automatización pueda quedar fuera del MVP.
- Logs sin exposición de información sensible innecesaria.

7. Integración con infraestructura HxL

La plataforma se desplegará en servidor de HxL y deberá convivir con el esquema existente basado en Traefik. La solución debe asumir reverse proxy externo y no exponer puertos directamente a Internet más allá de los estrictamente necesarios para integración con el proxy.

- El backend y frontend pueden compartir red Docker interna.
- Traefik enruta al frontend público del panel.
- El backend puede permanecer no expuesto públicamente y ser consumido por el frontend a través del mismo dominio o ruta interna según definición final.
- El contenedor de base de datos no debe exponerse a Internet.
- Se deben documentar labels o configuración equivalente para Traefik como anexo técnico de despliegue.

8. Entregables esperados por Antigravity

Módulo	Entregable mínimo	Incluye UI	Incluye API/DB
0	Repositorio funcional + Docker local + base app	Sí	Sí
1	Auth, usuarios, clases, organizador-clase	Sí	Sí
2	CRUD estudiantes + filtros + soft-delete	Sí	Sí
3	Cuotas configurables y pagos mensuales	Sí	Sí
4	Rifas integradas con pagos y fondo común	Sí	Sí
5	Libro financiero básico	Sí	Sí
6	Dashboard y proyecciones	Sí	Sí
7	Calendario y cantina	Sí	Sí
8	Logs y herramientas administrativas	Sí	Sí

9. Secuencia recomendada de implementación

1. Construir Módulo 0 completo y validar setup local antes de iniciar negocio.
2. Cerrar Módulo 1 y Módulo 2 antes de tocar pagos, para asegurar padrón y autenticación estables.
3. Implementar Módulo 3 y Módulo 4 juntos o con acoplamiento controlado, dado que cuotas y rifas se cruzan de forma directa.
4. Sumar Módulo 5 una vez existan datos reales para validar conciliación.
5. Agregar Módulo 6 después de tener modelos y flujos estables; no antes.
6. Incorporar Módulo 7 cuando las operaciones de cantina y fechas críticas ya puedan nutrirse desde el panel.
7. Completar Módulo 8 antes del uso intensivo en producción.

10. Riesgos de implementación a evitar

- No modelar clase y turno correctamente, generando ambigüedad con nombres repetidos.
- Acoplar demasiado frontend y backend mediante respuestas ad hoc difíciles de mantener.
- Diseñar el módulo de rifas como si existiera una numeración unitaria por ticket, cuando el proceso real opera por hojas y cantidades.

- Permitir que el excedente de rifas se aplique automáticamente a meses siguientes sin respaldo de negocio.
- Mezclar el libro financiero con lógica de pago de cuotas de forma inseparable.
- Intentar introducir permisos finos o features extra antes de consolidar el flujo principal.
- Subestimar la importancia del historial y permitir borrados duros.

11. Definición de cierre del MVP

Se considerará que el MVP está terminado cuando un organizador pueda autenticarse, consultar el padrón de estudiantes, registrar cuotas y rifas de un mes, visualizar saldo y desgloses financieros, usar el calendario de tareas y revisar logs de acciones, todo dentro del mismo sistema, con persistencia estable en MariaDB, despliegue funcional bajo Docker y documentación suficiente para reinstalación y mantenimiento básico.

Criterio final para Antigravity

Cada módulo debe entregarse con código funcional, estructura de datos asociada, vistas mínimas operativas y criterio de aceptación cumplido. La prioridad no es producir pantallas decorativas sino flujos consistentes, auditables y utilizables por organizadores reales.